

Analys och förutsägelse av fotbollsmatcher: En maskininlärning med Neural network och Random Forest

Quang Dai Daniel Bui

Kurs
Pythonprogrammering för AI-utveckling HT24
IT - Högskolan Göteborg

5 december 2024

Innehåll

1	Inledning	3
2	Mål	3
3	Teori	3
4	Metod/Experiment/Utförande	4
4.1	Datainsamling och preprocessing	4
4.2	Modellering	5
4.3	Utvärdering	6
4.4	Visualisering	6
5	Resultat	6
5.1	Neural Network	6
5.2	Random Forest	6
5.3	Visualiseringar	7
6	Diskussion	7
6.1	Random forest	8
6.2	Neural Network	8
6.2.1	Möjliga orsaker	9
6.3	Jämförelse	9
7	Slutsatser	9
7.1	Neural Network	9
7.2	Random Forest	9
7.3	Slutsats	10

1. Inledning

Syftet med projektet är att analysera och förutsäga fotbollsmatcher såsom i La Liga säsongen 2023-2024 genom att använda och kombinera data från två olika källor (Footystats och Kaggle). Syftet med projektet är att bedöma hur väl specifika faktorer som skottstatistik och bollinnehav kan användas för skapa modell och förutse lagets prestationer. Projektet uppnås genom att ha lösa genom regressor och implementera maskinlärningsmodeller. Två stycken modeller kommer skapas: Neural Network och Random Forest regressor.

2. Mål

Målet är att

1. Samla och kombinera data från två olika källor för få fler faktorer och integrera relevanta egenskaper för analys.
2. Skapa och träna maskinlärningsmodeller för förutsäga antal skott för specifikt hemmalag.
3. Jämföra modellernas prestanda och tolka resultaten.

3. Teori

Genom att förutsäga skottstatistik används två metoder inom maskininlärning:

- Neural Network: Modell som simulerar hjärnans funktioner genom att ha flera lager av neuroner. Lämpligt att lösa icke-linjära problem.
- Random Forest: Ensemblemodell som bygger flera beslutsträd och få resultaten för robusta förutsägelser. Lämpligt för strukturerade data och problem med många kategoriska och numeriska egenskaper.

Projektet kommer data processas genom normalisering, hantering av null värden och One-hot Encoding för att förbättra accuracy.

4. Metod/Experiment/Utförande

Projektet kommer utföra 4 metoder där data används från filen `spain-la-liga-matches-2023-to-2024-stats.csv` och `la_liga_results_2324.csv`.

4.1 Datainsamling och preprocessing

- Data laddades från två csv filer och använder `spain-la-liga-matches-2023-to-2024-stats.csv` bollinnehav som feature och kombinerar data till `la_liga_results_2324.csv` där man sammanslogs baserat på datum och lagnamn. Datan sparas till en ny CSV fil med namnet `fixed_merged_la_liga_results.csv`.

```
# Mapping för namn differenser i de .csv filerna
team_name_mapping = {
    'Athletic Club': 'Athletic Club Bilbao',
    'Osasuna': 'CA Osasuna',
    'Getafe': 'Getafe CF',
    'Girona': 'Girona FC',
    'Granada': 'Granada CF',
    'Las Palmas': 'UD Las Palmas',
    'Mallorca': 'RCD Mallorca',
    'Sevilla': 'Sevilla FC',
    'Valencia': 'Valencia CF',
}

# Normalisera lagnamn med mapping
footstats_df['home_team_name'] = footstats_df['home_team_name'].replace(team_name_mapping)
footstats_df['away_team_name'] = footstats_df['away_team_name'].replace(team_name_mapping)
kaggle_df['hometeam'] = kaggle_df['hometeam'].replace(team_name_mapping)
kaggle_df['awayteam'] = kaggle_df['awayteam'].replace(team_name_mapping)

# Column för features och läggs till possession
merge_columns = ['date', 'home_team_name', 'away_team_name', 'home_team_possession', 'away_team_possession']

# Välj och byts ut namnet
footstats_df_subset = footstats_df[merge_columns].rename(
    columns={
        'home_team_name': 'hometeam',
        'away_team_name': 'awayteam',
    }
)

# Merging datumen samt lagnamnet
merged_df = pd.merge(
    kaggle_df,
    footstats_df_subset,
    on=['date', 'hometeam', 'awayteam'],
    how='left'
)
```

Figur 1: Skapar en mapping för rätta lagets namn genom att byta ut namnet från ena CSV till andra CSV

- Kategoriska variabler kodades om med One-Hot Encoding och löser ut alla null värden som finns i kolumnen.
- Data normaliserades med StandardScaler från SKLearn.

```
#Class som förbereder data och kolla null värden
class DataPreparation:
    def __init__(self, merged_csv):
        self.scaler = StandardScaler()
        self.df = pd.read_csv(merged_csv)
        #One hot coding
        def onehot(self):
            self.df = pd.get_dummies(self.df)

            X = self.df.drop('home_total_shots', axis=1)
            y = self.df['home_total_shots']
            X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

            #Lös alla null i .csv
            X_train = X_train.fillna(X_train.mean())
            X_test = X_test.fillna(X_test.mean())
            #Försäkra X_train och X_test shape
            X_test = X_test.reindex(columns=X_train.columns, fill_value=0)

            X_train_scaled = pd.DataFrame(self.scaler.fit_transform(X_train), columns=X_train.columns)
            X_test_scaled = pd.DataFrame(self.scaler.transform(X_test), columns=X_test.columns)

            return X_train_scaled, X_test_scaled, y_train, y_test
```

Figur 2: Klassen hanterar dataförberedelser inför maskininlärning. Den tar in en CSV-fil, hanterar null värden, preprocessar data och delar upp den i tränings- och testuppsättningar och Scaling.

4.2 Modellering

- Två modeller tränades: Ett Neural network (Tensorflow) och Random Forest (med hyperparameteroptimering via GridSearchCV).

```
#class for Neural Network modellen
class NeuralNetwork:
    def __init__(self):
        #Skapa modellen
        self.model = tf.keras.Sequential()
        #Dropout och BatchNormalization förbättring för accuracy
        tf.keras.layers.BatchNormalization(),
        tf.keras.layers.Dense(128, activation='relu'),
        tf.keras.layers.Dense(64, activation='relu'),
        tf.keras.layers.Dropout(0.2),
        tf.keras.layers.Dense(10)
        #Kraftfäst på learning rate, dropout layer, kolla early stopping
        self.model.compile(optimizer=tf.keras.optimizers.Adam(learning_rate=0.001),
                           loss='mean_squared_error',
                           metrics=['mae'])

    #Träna
    def train(self, X_train, y_train):
        #Earlystopping for accuracy improvement
        callback = [
            tf.keras.callbacks.EarlyStopping(monitor='val_loss', patience=10, restore_best_weights=True),
            tf.keras.callbacks.ReduceLROnPlateau(monitor='val_loss', factor=0.5, patience=5)
        ]
        history = self.model.fit(X_train, y_train, validation_data=None,
                                epochs=200, callbacks=callback, verbose=1)
        return history
```

Figur 3: Neural Network klassen skapar modellen med ReLU som aktiveringsfunktion och kompilerar med optimerare Adam med inlärningshastighet på 0,001

- Data delades upp i tränings- tests sets (70/30).

```

#class for Random Forest model
class RandomForest:
    def __init__(self):
        #Hyperparameter grid
        self.param_grid = {
            'n_estimators': [100, 200, 300],
            'max_depth': [None, 10, 20, 30],
            'min_samples_split': [2, 5, 10],
            'min_samples_leaf': [1, 2, 4]
        }
        self.grid_search = None

    #fits model
    def train(self, X_train, y_train):
        rf = RandomForestRegressor(random_state=42)
        self.grid_search = GridSearchCV(estimator=rf, param_grid=self.param_grid, cv=5, n_jobs=-1, verbose=2, scoring='r2')
        self.grid_search.fit(X_train, y_train)
        return self.grid_search

    #fits model and predicts
    def evaluate(self, X_test, y_test):
        if self.grid_search is not None:
            best_model = self.grid_search.best_estimator_
            y_predict = best_model.predict(X_test)
            mse = mean_squared_error(y_test, y_predict)
            print("Random Forest- MSE: %s" % mse)
        else:
            print("WARNING: Train the model first!")

```

Figur 4: Random forest modellen skapas med hjälp av GridSearchCV för optimera hyperparametrar

4.3 Utvärdering

Modellerna utvärderas med matriser som Mean Squared Error (MSE), Mean Absolute Error (MAE) och R square (R^2).

4.4 Visualisering

Programmet kommer visa två visualisering för modellens prestanda. Modellförluster (loss) och testresultat som visualiseras med hjälp av kurvor, colormaps, och scatter plots

5. Resultat

Modellens resultat för Neural Network och Random forest

5.1 Neural Network

- Förlust (loss): 17.44
- Mean Absolute Error (MAE): 3.27
- R square (R^2): 0.45

5.2 Random Forest

- Bästa parametrar: 'max depth': None

'min samples leaf': 1

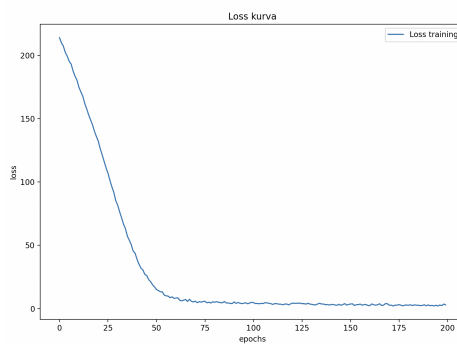
'min samples split': 5

'n estimators': 200

- Train Accuracy: 0.912
- Test Accuracy (R^2): 0.516
- Prediktioner var mer träffsäker jämfört med Neural Network modellen

5.3 Visualiseringar

- Loss för Neural network visar att modellen tränas i rätt fas och loss värden går ner ju mer epochs körs

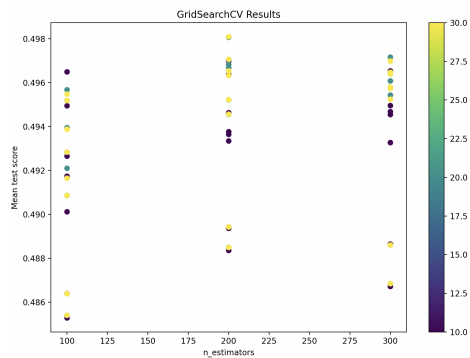


Figur 5: *Neural Network visualisering på Loss training mellan Loss och epochs*

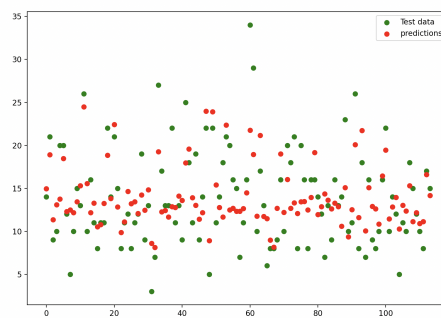
- GridSearchCV för Random forest visade resultaten för effekten av hyperparametrar på modellens prestanda.
- Scatter-plottar visade skillnaden mellan förutsägelser och verkliga värden för både modeller.

6. Diskussion

Två modellerna visade positiva resultat men inte exakta resultat från prediktioner jämfört med verkliga resultatet.



Figur 6: *Random Forest* visualisering på vilka parametrar är bäst utav *GridSearchCV*



Figur 7: *Random Forest* visualisering på prediktiva och test data

6.1 Random forest

Random forest modellen visade bättre prestanda jämfört med Neural Network på grund av den relativt datamängden och features som analyserades. Det tyder på att modellen utnyttjade de strukturella egenskaperna väl och kan hantera både linjära och icke-linjära data utan att kräva mycket dataträning.

6.2 Neural Network

Neural Network modellen prestandan var mindre sämre än väntat, med förlust på (17,44), ett relativt litet fel i MAE (3,27), och ett R square värde på (0,45). Detta visar att det neural network modellen presterar sämre än Random forest

6.2.1 Möjliga orsaker

- Modellen kan ha varit grundläggande för datasetet som kan leda till underanpassning
- Hyperparametrarna som arkitekturen kan ha behövt finjusteras.
- Mängden datan från CSV och bristen på features kan ha begränsat modellens förmåga att förutse rätt resultat.

6.3 Jämförelse

Jämfört med Random forest är det möjligt att det Neural network inte är lämpligt i detta fall, särskilt på strukturerade data som kan hanteras bättre av enklare modeller som Random Forest.

7. Slutsatser

7.1 Neural Network

Resultaten för neural network visar att modellen var inte helt lämpligt designad och tränad för projektet och dataset. Sämre R square värde visar att modellen inte lyckas att representera förhållandet mellan variablerna och målvariabeln. För att förbättra prestandan behövs mycket mer data insamling med flera features för att få en lämpligt resultat från Neural Network modellen

7.2 Random Forest

Random forest gav ett bättre resultat för detta dataset, både när det gäller predaktiv noggrannhet och användarevänligt. Detta beror främst på hantering mot små dataset och förmåga att utnyttja strukturerade egenskaper effektivt. För de flesta resultat skiljer runt 3 skott från test datan. Enstaka resultat som skiljer stort så kan det hända exempelvis att nyckelspelaren hos ett lag inte spelades under matchen eller någon av lagen har fått rött kort, vilket leder till ganska hög volatilitet på statistiken under matchen.

7.3 Slutsats

Med strukturerade data och liten datamängd så är det lämpligare att arbeta modeller som Random Forest än Neural network. Om det fanns fler features som exempelvis nyckelspelare hos ett lag och större datamängder från fotbollmatcher som samlas in, kan Neural Network modellen vara ett alternativ.